Heap Management

Acknowledgement

- Alfred V Aho, Monica S. Lam, Ravi Sethi, Jeffrey D Ullman- “Compilers- Principles, Techniques and Tools”

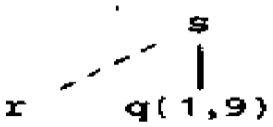
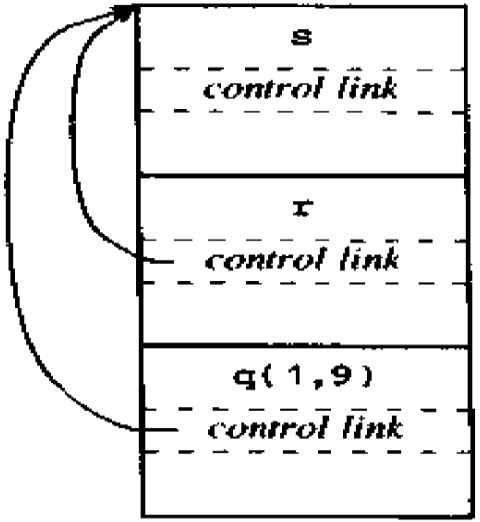
Heap Management

- The heap is the **portion of the storage** that is used for data that **lives indefinitely**, or until the program explicitly deletes it.
- Local variables typically become inaccessible when their procedures end
- But objects or other data whose existence is not tied to the procedure activation that creates them i.e. creation through ***new*** in C++ or Java are stored on a heap

Heap Allocation

- Stack allocation **cannot be used** if:
  - **Local vars** to be **retained** after activation ends
  - A called activation **outlives the caller**
  - Deallocation of **activation records need not occur in LIFO fashion**
- Heap allocation: **parcels out pieces of contiguous storage**, as needed
  - Can be deallocated in any order

Heap Allocation

Position in the Activation Tree	Activation Records in the Heap	Remarks
		Retained activation record for r

The Memory Manager

- Keeps track of all the **free space** in heap storage
- It performs **two basic functions**:
  - Allocation
  - Deallocation

The Memory Manager

- **Properties** of memory managers :
  - **Space Efficiency**: A memory manager should **minimize the total heap space** needed by a program
  - **Program Efficiency**: A memory manager should make **good use of the memory subsystem** to allow programs to run faster
  - **Low Overhead**: memory **allocations and deallocations** are frequent operations in many programs, it is important that these **operations be as efficient as possible**.

The Memory Hierarchy of a Computer

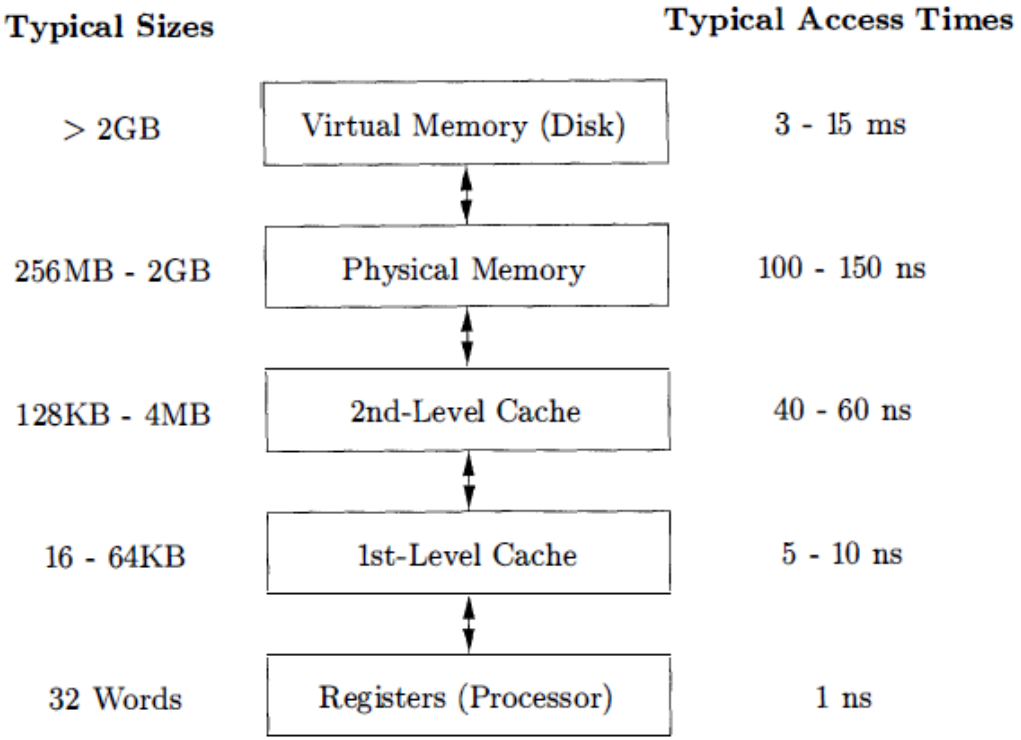


Figure 7.16: Typical Memory Hierarchy Configurations

Locality in Programs

- Most programs exhibit a **high degree of locality**; that is, they **spend** most of their **time** executing a relatively **small** fraction of the **code** and touching only a **small fraction** of the **data**.
- A program has **temporal locality** if the **memory locations** it accesses are likely to be **accessed again within a short period of time**.
- A program has **spatial locality** if **memory locations close to the location** accessed are likely also to be **accessed within a short period of time**.

Locality in Programs

- Place the most **common instructions and data** in the **fast-but-small storage**, while leaving the rest in the slow-but-large storage
- The fastest cache often is **not large**
- Therefore **adjust** the contents of the fastest storage **dynamically** and use it to **hold instructions** that are likely to be **used heavily** in the near future

Optimization Using the Memory Hierarchy

- Most **recently used instructions** in the cache
- When a **new instruction** is executed, there is a **high probability** that the **next instruction** also will be executed.
- To improve the **spatial locality** of instructions is to have the compiler place **basic blocks** that are **likely to follow** each other **contiguously** - on the **same page**, or even the **same cache** line

Optimization Using the Memory Hierarchy

- Improve the temporal and spatial locality of data accesses in a program by **changing the data layout or the order of the computation**
- For programs that visit **large amounts of data repeatedly**, **bring some data from a slow level** of the memory hierarchy **to a faster level**

Reducing Fragmentation

- **Holes**, the free chunks of memory need not reside in a contiguous area of the heap.
- Coalesce (combine) **contiguous holes into larger holes**
- The memory may end up getting fragmented, consisting of **large numbers of small, noncontiguous holes**.

Best- Fit and Next-Fit Object Placement

- Good strategy for minimizing fragmentation
  - **best-fit:** allocate the requested memory in the smallest available hole that is large enough.
  - **first-fit:** placed in the first hole in which it fits, takes less time to place objects, but has been found inferior to best-fit in overall performance.
- best-fit may not be the best in terms of spatial locality.
- **next-fit strategy**, try to allocate the object in the chunk that has **last been split, whenever enough space for the new object remains in that chunk.**
- **Next-fit** also tends to **improve the speed of the allocation** operation.

Managing and Coalescing Free Space

- When an object is **deallocated manually**, **combine** (coalesce) that chunk **with adjacent chunks** of the heap, to form a larger chunk
- **bin** for chunks: (**not to coalesce adjacent blocks**)
  - **All the chunks of one size**.
  - keep a **bitmap**, 1 indicates the chunk is occupied; 0 indicates it is free.
  - When deallocated, change its bit 1 to a 0.
  - For allocation, **find any chunk with a 0 bit, change that bit to a 1**, and use the corresponding chunk.
  - If there are **no free chunks, get a new page, divide it into chunks** of the appropriate size

Managing and Coalescing Free Space

- When the heap is managed as a whole, there are **two data structures** that are useful **to support coalescing of adjacent free blocks**:
- **Boundary Tags**:
  - At **both the low and high ends of each chunk, keep a free/used bit** that tells whether or not the block is currently allocated (used) or available (free) .
  - **Adjacent to each free/used bit is a count of the total number of bytes** in the chunk.
- **Doubly Linked, Embedded Free List**:
  - The free chunks (but **not the allocated chunks**) are also linked in a **doubly linked list**.
  - The **pointers for this list are within the blocks themselves**, say adjacent to the boundary tags at either end.

Manual Deallocation Requests

- Programmer must **explicitly** arrange for the **deallocation** of data
- Ideally, any storage that will no longer be accessed should be deleted

Problems with Manual Deallocation

- Manual memory management is error-prone.
  - failing ever to delete data that **cannot be referenced** is called a **memory leak error**
    - memory leaks may **slow down the execution** of a program due to increased memory usage
    - **do not affect program correctness**
    - **Automatic garbage collection** gets rid of memory leaks
    - **garbage collection program** may still **use more memory** than necessary
  - referencing deleted data is a **dangling-pointer-dereference error**
    - dereferencing a dangling pointer **after the freed storage is reallocated** almost always creates a **program error** that is hard to debug
    - programming error is to **access an illegal address**

Programming Conventions and Tools

- **Object ownership**
  - useful when an object's **lifetime** can be statically reasoned about
  - **Associate an owner with each object**
  - **The owner is a pointer to that object**
  - The owner (i.e., its function) is **responsible** for either **deleting** the object or for **passing** the object to another owner

Programming Conventions and Tools

- **Reference counting**
  - useful when an object's lifetime needs to be determined dynamically
  - **associate a count** with each dynamically allocated object
  - When the count goes to **zero**, the object can no longer be referenced and can therefore **be deleted**.
  - Reference counting is **expensive** because it **imposes an overhead** on every operation that stores a pointer

Programming Conventions and Tools

- **Region-based allocation**
  - useful for collections of objects whose **lifetimes are tied to specific phases** in a computation.
  - When objects are created to be used only within some step of a computation, we can **allocate all such objects in the same region**.
  - We then **delete the entire region** once that computation step completes.
  - region-based allocation technique has **limited applicability very efficient**

Introduction to Garbage Collection

- Data that **cannot be referenced** is generally known as garbage
- The garbage collector finds these unreachable objects and **reclaims their space** by handing them to the memory manager, which keeps track of the free space

Design Goals for Garbage Collectors

- A Basic Requirement : Type Safety
  - A language in which the **type of any data** component can be determined is said to be type safe.
  - In **Java**, types cannot be determined at compile time, but can be **determined at run time**.
  - In **unsafe languages, memory addresses can be manipulated arbitrarily**: arbitrary arithmetic operations can be applied to pointers to create new pointers, and arbitrary integers can be cast as pointers

Design Goals for Garbage Collectors

- Performance Metrics
  - Overall Execution Time
  - **Space Usage**: It is important that garbage collection avoid fragmentation and make the best use of the available memory
  - **Pause Time**: Simple garbage collectors are notorious for causing programs to pause suddenly for an extremely long time, as garbage collection kicks in without warning
  - **Program Locality**: it can improve the program's spatial locality by relocating data used together in the same cache or pages

Reachability

- All the data that can be accessed directly by a program, without having to dereference any pointer, as the **root set**.
- A program obviously can **reach any member** of its root set at any time.
- Recursively, **any object** with a reference that is **stored in the field members or array elements of any reachable object** is **itself reachable**.

Reachability

- The set of **reachable objects changes as a program executes**
- **Four basic operations** that a mutator performs **to change** the set of reachable objects:
  - Object Allocations:
  - Parameter Passing and Return Values
  - Reference Assignments
  - Procedure Returns
- **New objects** are introduced through object allocations.
- Parameter passing and assignments can **propagate reachability**
- Assignments and ends of procedures can **terminate reachability**
- As an object becomes unreachable, it can cause **more objects to become unreachable**

Reference Counting Garbage Collectors

- Every object must have a field for the **reference count**
- Reference counts can be maintained
  - **Object Allocation**: The reference count of the **new object is set to 1**.
  - **Parameter Passing**: The reference count of each **object passed** into a procedure is **incremented**.
  - **Reference Assignments**: For statement u = v, the reference count of the object referred to by **v goes up** by one, and the count for the old object referred to by **u goes down** by one.
  - **Procedure Returns**: All the references held by the **local variables** of that procedure activation record must be **decremented**.
  - **Transitive Loss of Reachability**: Whenever the reference count of an object becomes zero, we must also decrement the count of each object pointed to by a reference within the object.
- it cannot collect unreachable, cyclic data structures, and it is expensive.

Reference Counting Garbage Collectors

- The overhead of reference counting is **high**
- **Deferred reference counting** has been proposed as a means to eliminate the overhead associated with updating the reference counts due to local stack accesses
  - reference counts **do not include references from the root set** of the program.
  - **An object is not considered to be garbage until the entire root set is scanned and no references to the object are found.**

Reference Counting Garbage Collectors

- Advantage:
  - Garbage collection is performed in an **incremental fashion**
  - Garbage is collected **immediately**, keeping **space usage low**